



WIF3009

Python for Scientific Computing

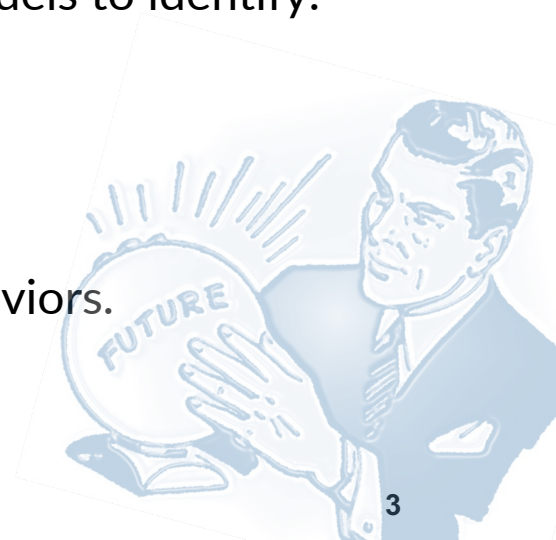
Applied Machine Learning



Intro to Predictive Analytics

Predictive Analytics (what is ?)

- Predictive analytics is a branch of data analysis that uses historical data and statistical algorithms to make predictions about future events or trends.
- It involves the application of various techniques and models to identify:
 - Patterns
 - Relationships
 - Associations within data
- Use the knowledge to forecast future outcomes or behaviors.



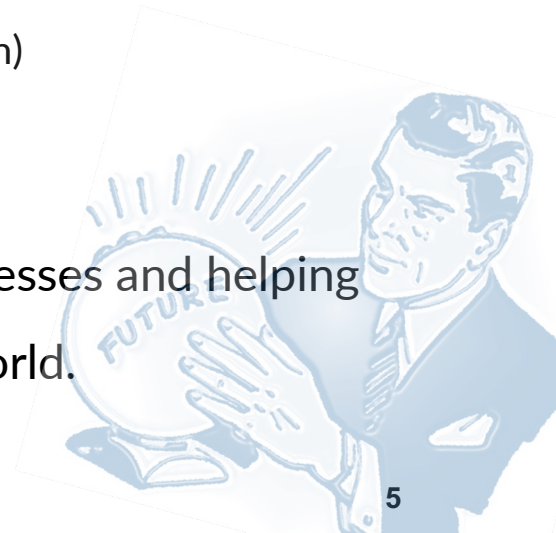
Predictive Analytics (what is ?)

- In essence, predictive analytics leverages data-driven insights to answer questions like:
 - What is likely to happen in the future?
 - What are the potential outcomes or scenarios based on existing data?
 - How can businesses and organizations make informed decisions and take proactive actions to optimize results?



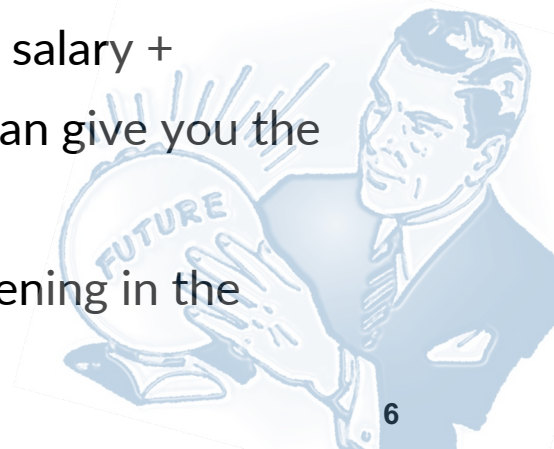
Predictive Analytics (what is ?)

- Predictive analytics is widely used across various industries and applications, including:
 - Finance (credit scoring and fraud detection)
 - Marketing (customer segmentation and campaign optimization)
 - Healthcare (disease prediction and patient outcomes)
 - Manufacturing (quality control and supply chain optimization)
- It plays a crucial role in enhancing decision-making processes and helping organizations stay competitive in today's data-driven world.



Predictive Analytics (what is ?)

- A predictive model is done by utilizing **useful features** from the data set.
- **Features** or '**columns**' in the data set is fed to the model. The model then performs its magic before producing the value/result for the dependent variable/our target variable.
- In simpler words, for example, processing an employee's salary + satisfaction score + promotion/raise in the last 5 years can give you the risk score for the employee to resign.
- It is sufficient to know at this level that the 'magic' happening in the learning model is a bunch of mathematical functions.



Predictive Analytics (what is ?)

- There are various predictive algorithms that can be implemented. Examples include:
 - Linear Regression
 - Logistic Regression
 - Support Vector Machines
 - Clustering methods
 - Neural Networks



Predictive Analytics (what is ?)



- UK's emergency response team would benefit tremendously if they can foresee the likelihood of accidents occurring at a certain place.
- Based on the road safety dataset, you can build a learning model that accurately predicts the chances of accidents happening at specific time or weather.
- It can help in improving response time and resource allocation.

Predictive Analytics (what is ?)



- It is a tremendous advantage if a company can foresee the tendency to resign among its employees before they actually happen.
- Based on the employee dataset, a company can accurately predict if an employee is at a high risk of resignation.
- This is important for the company to be proactive and provide appropriate benefits or counter offer so the employee chooses to stay.

Predictive Analytics (case study) – Cambridge Analytica



Interview: Trump's chief data scientist reveals the data approach that won an election

Chief data scientist for the Trump presidential election campaign and Cambridge Analytica exec shares how first-party data was instrumental and why programmatic advertising is dying

NADIA CAMERON (CMO) 06 APRIL, 2017 11:04

[Share](#) [Like 16](#) [Tweet](#) [Print](#)



Matt Oczkowski, Cambridge Analytica

The New York Times

How Trump Consultants Exploited the Facebook Data of Millions

By Matthew Rosenberg, Nicholas Confessore and Carole Cadwalladr

March 17, 2018



[Leer en español](#)

(After this story was published, Facebook came under harsh criticism from lawmakers in the United States and Britain. [Read the latest.](#))

LONDON — As the upstart voter-profiling company [Cambridge Analytica](#) prepared to wade into the 2014 American midterm elections, it had a problem.

The firm had secured a \$15 million investment from [Robert Mercer](#), the wealthy Republican donor, and wooed his political adviser, Stephen K. Bannon, with the promise of tools that could identify the personalities of American voters and influence their behavior. But it did not have the data to make its new products work.

Predictive Analytics (case study) - UPS

- United Parcel Service (UPS), the biggest package shipping company in the world spends over \$1 billion on Big Data Analytics.
- In one of the most interesting use case of prescriptive analytics, UPS truck drivers (almost) never make left turns in their driving
- (Ref: <http://edition.cnn.com/2017/02/16/world/ups-trucks-no-left-turns/index.html>)
- Using prescriptive analytics, UPS saves 10 million gallons of fuel a year 100,000 metric tons of CO₂ (equivalent to emission of 21000 cars), and 6 – 8 fewer miles per route.





DECISION MAKING & ML USING PYTHON

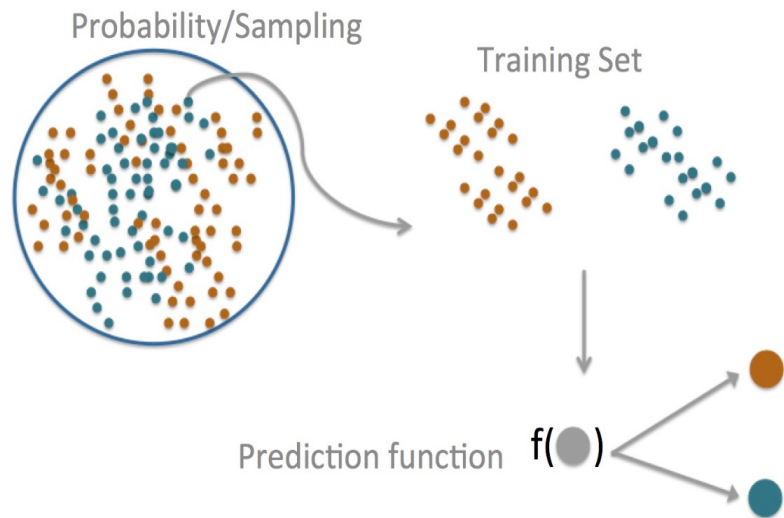
Introduction (what is ?)



- One of the most common tasks performed by data scientists and data analysts are prediction and machine learning.
- This will cover the basic components of building and applying prediction functions with an emphasis on practical applications.
- We also provide basic grounding in concepts such as training and tests sets, overfitting, and error rates.
- Also, introducing a range of model based and algorithmic machine learning methods including regression, classification trees, Naive Bayes, and random forests.
- This topic will cover the complete process of building prediction functions including data collection, feature creation, algorithms, and evaluation.

The Central Dogma of Prediction

- Suppose, we want to predict for these dots whether they're red or blue.
- Have a big group of dots that we want to predict about, and then use probability and sampling to pick a training set.
- The training set will consist of some red dots and some dots, and measure the whole bunch of characteristics of those dots.
- Then use those characteristics to build what's called a prediction function, and the prediction function will take a new dot, whose color we don't know, but using those characteristics we measured will predict whether it's red or whether it's blue.
- Then we can go off and try to evaluate whether that prediction function works well or not.



Relative Order of Importance (Garbage in = Garbage Out)



question -> **input data** -> features -> algorithm -> parameters-> evaluation

- May be easy (movie ratings -> new movie ratings)
- May be harder (gene expression data -> disease)
- Depends on what is a “good prediction”.
- Often (**more data > better models**)

Relative Order of Importance (Features matter!)



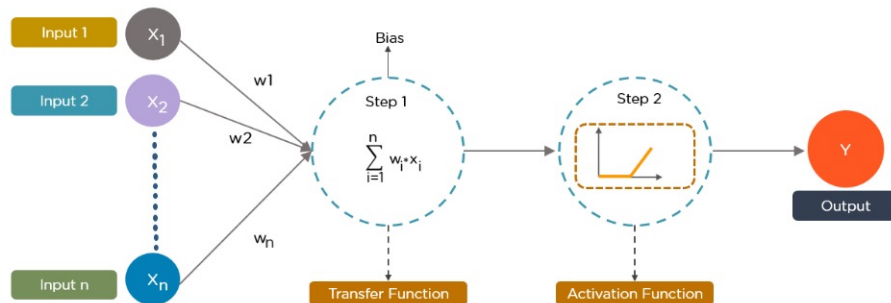
question -> input data -> **features** -> algorithm -> parameters-> evaluation

- Properties of good features:
 - Lead to data compression
 - Retain relevant information
 - Are created based on expert application knowledge
- Common mistakes:
 - Trying to automate feature selection
 - Not paying attention to data-specific quirks
 - Throwing away information unnecessarily

Relative Order of Importance (Algorithm)

question -> input data -> features -> **algorithm** -> parameters-> evaluation

- Algorithm actually matter less than you'd think (surprise!).
- Using a very, sensible approach will get you a very large weight of solving the problem.
- Getting the absolute best method can improve, often doesn't improve that much over, sort of, most good sensible methods.



Issues and Tradeoff



- Issues to consider when building a prediction model:
 - Interpretability
 - Accuracy
 - Performance
 - Simplicity
 - Scalability
- Prediction is about accuracy tradeoffs.
 - Interpretability versus accuracy
 - Speed versus accuracy
 - Simplicity versus accuracy
 - Scalability versus accuracy

Prediction Study Design



1. Define the error rate.
2. Split data into:
 - Training, Testing, Validation (optional)
3. On the training set, select features.
 - Use cross-validation.
4. On the training set, choose the prediction function.
 - Use cross-validation.
5. If no validation set, apply 1x to test set.
6. If have validation set,
 - Apply to validation set and refine.
 - Apply 1x to test set.

Basic Terms

- In general, Positive = identified and Negative = rejected. Therefore:
 - True positive = correctly identified
 - False positive = incorrectly identified
 - True negative = correctly rejected
 - False negative = incorrectly rejected
- Medical testing example:
 - True positive = Sick people correctly diagnosed as sick.
 - False positive = Healthy people incorrectly identified as sick.
 - True negative = Healthy people correctly identified as healthy.
 - False negative = Sick people incorrectly identified as healthy.

Classification Outcome

	GOLD STANDARD	
	Female (Positive)	Male (Negative)
Female (Positive)	True Positives (TP) = 41	False Positives (FP) = 4
Male (Negative)	False Negatives (FN) = 9	True Negatives (TN) = 46

Types of Prediction



Types of Prediction (Classification)



- The main goal of **classification** is to predict the target class (True/False).
- Binary Classification:
 - If the trained model is for predicting any of two target classes. It is known as binary classification.
 - Considering the student profile to predict whether the student will pass or fail.
 - Example: Status (Pass/Fail)
- Multiclass Classification:
 - If we have to predict more than two target classes it is known as multi-classification.
 - Considering all subject details of a student to predict the grade of the students for a specific subject.
 - Example: Student grade (A/B/C/D/E/F)

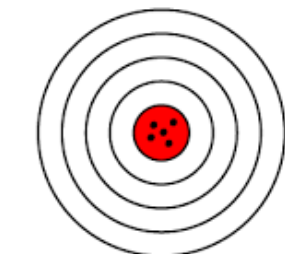
Types of Prediction (Regression)



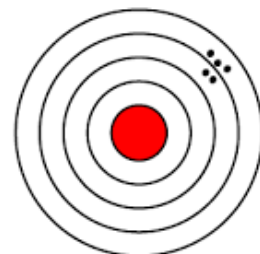
- The main goal of **regression** algorithms is to predict the discrete or a continuous value.
- In some cases, the predicted value can be used to identify the linear relationship between the attributes.
- Suppose the increase in the product advantage budget will increase the product sales.
- Based on the problem difference regression algorithms can be used.
- Some of the regression algorithms are linear regression, polynomial regression, support vector regression.
- Example: Predicting the value of MYR currency for the next coming day based on several indicators and features.

Common Error Measures

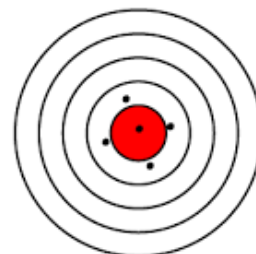
- Mean squared error (or root mean squared error)
 - Continuous data, sensitive to outliers.
- Mean absolute deviation
 - Continuous data, often more robust.
- Sensitivity (recall)
 - If you want few missed positives.
- Specificity
 - If you want few negatives called positives.
- Accuracy
 - Weight false positives/negatives equally.
- Concordance
 - One example is Kappa value



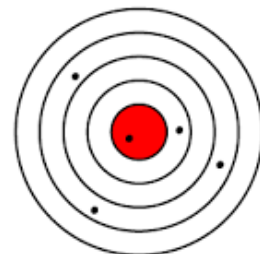
Accurate & Consistent



Not Accurate
but Consistent



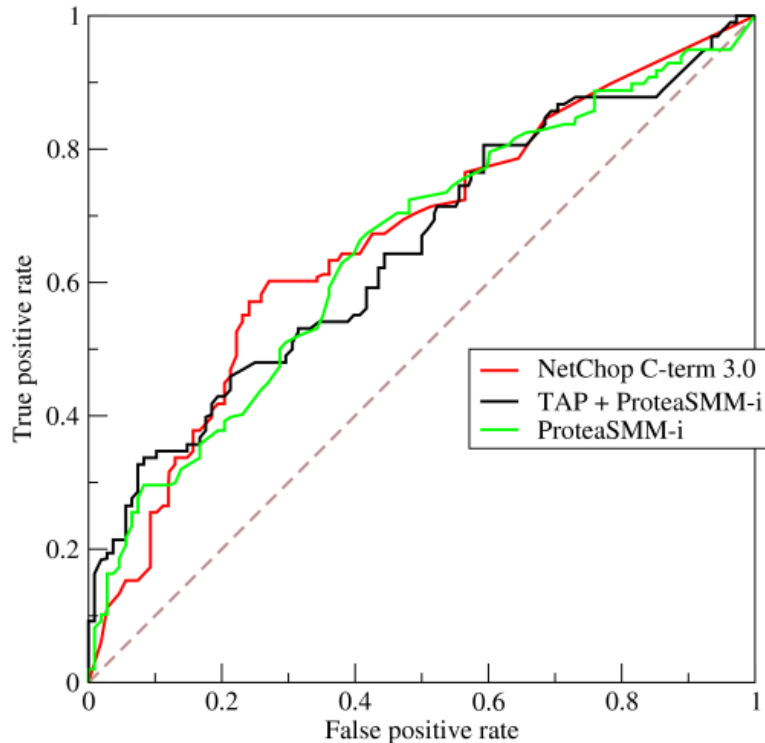
Accurate but
Not Consistent



Not Accurate &
Not Consistent

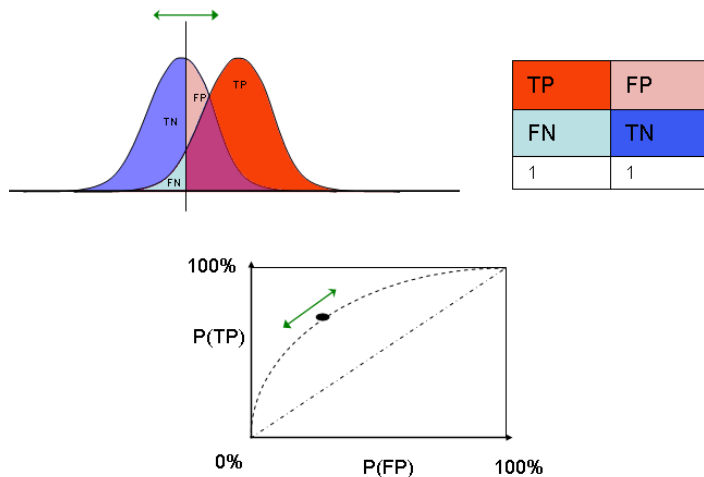
Receiver Operating Characteristic (ROC) curves

- A very commonly used technique to measure the quality or goodness of a prediction algorithm.
- In a binary classification, the cut we choose gives different results.
- In other words, if we predict a probability that we're going to be alive is 0.7 and we say all the people above 0.5 will be assigned to be alive, then that's one prediction algorithm.
- So, for example, if we look at the NetChop algorithm. And we look at 1 minus the specificity is equal to 0.2 in other words this specificity is quite high, it's 0.8.
- We can trace that number up and we can get about a point on the curve the red curve in this case and that corresponds to a sensitivity to only about 0.4. So it's very highly specific but not very sensitive.



Area Under the Curve (AUC)

- Although the curve tells a little bit about the tradeoffs, we may actually want to know which of these prediction algorithms is better.
- One way to quantify one curve versus the other, is to calculate the area under the curve (AUC).
- That area quantifies how good, that particular prediction algorithm is.
- **AUC = 0.5** : Random guessing
- **AUC = 1.0** : Perfect classifier
- In general, AUC above 0.8 is considered “good”.

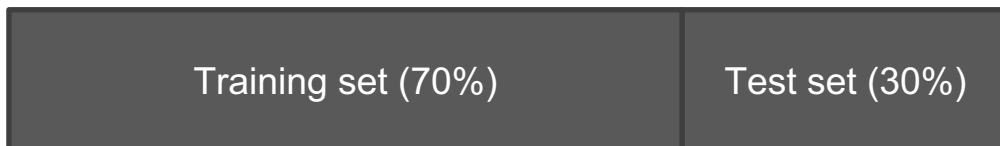


Datasets Splitting



- In machine learning, the dataset need to be split into multiple parts for several purposes.
- The commonly used approach is the 70/30 % split:
 - Training set: The data used to train the predictive model.
 - Test set: The data used to evaluate the performance of the predictive model on unseen data.
- Sometimes there is also validation set used for model tuning however it is optional.
- Validation set comes from the training set.

Dataset



Sample Errors

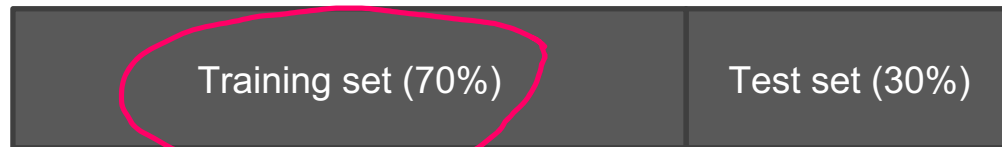


- ***In Sample Errors***: The error rate you get on the same data set you used to build your predictor. Sometimes called resubstitution error.
- ***Out of Sample Errors***: The error rate you get on a new data set. Sometimes called generalization error.
- Key ideas:
 - Out of sample errors is what we need to look out for.
 - In sample error < out of sample error.
 - The reason is overfitting.
 - Matching the algorithm to the data you have.

Sample Errors – In Sample Errors

- ***In Sample Errors*** uses the training set to evaluate the performance of the prediction model.
- Firstly, train the predictive model using the training set.
- Next, test the predictive model using the same data set as before (training set).
- The error rate from the prediction results is the ***in sample errors***.
- This method is useful to assess the initial predictive model ability by using the same data as during the model training.

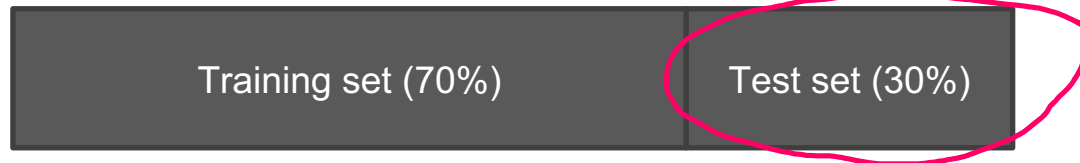
Dataset



Sample Errors – Out of Sample Errors

- **Out of Sample Errors** uses the test set to evaluate the performance of the prediction model.
- Firstly, train the predictive model using the training set.
- Next, test the predictive model using the test set (sometimes people called it hidden set).
- The error rate from the prediction results is the **out-of-sample errors**.
- This method is used to assess the ability of the predictive model on new data (unseen data).

Dataset



Overfitting and Underfitting



- In machine learning, **overfitting** and **underfitting** are among the dilemma and concerns.
- Overfitting means you have **created an overly complex model** that instead of learning on the patterns in the data, the model also learns about the noise. In simpler terms, the model memorizes the data instead of learning it. It could lead to a false assumption that your model is performing really good.
- Underfitting is the opposite of overfitting. **Your model is too 'lose'** that it has not really learn anything from the data.
- Optimizing your model so it does not fall into this two categories is vital.

scikit-learn



- **scikit-learn** is a Python machine learning library that provides various learning algorithms including supervised and unsupervised learning.
- There are other libraries:
 - Pytorch
 - TensorFlow
 - Keras
- scikit-learn integrates well with Python's SciPy stack i.e. NumPy, Pandas

scikit-learn



- scikit-learn has a wide range of supervised learning algorithms.
- Among them are:
 - Generalized Linear Models:
 - Ordinary Least Squares (OLS)
 - Ridge Regression
 - Lasso Regression
 - Bayesian Regression
 - Support Vector Machine (SVM)
 - Nearest Neighbors
 - Decision Trees
 - Ensemble methods

scikit-learn



- scikit-learn has a repository of datasets taken from the UCI Machine Learning repository.
- You will be using the Boston Breast Cancer dataset.
- You need to import the `datasets` module from `sklearn` before you can use it.

```
In [581]: from sklearn import datasets

          cancer = datasets.load_breast_cancer()
          cancer.keys()

Out[581]: dict_keys(['data', 'target', 'target_names', 'DESCR', 'feature_names'])
```

```
In [583]: cancer.data.shape

Out[583]: (569, 30)
```

Data Splitting (Basics)

- `scikit-learn.model_selection` module provides the `train_test_split` submodule to simplify this process.
- The `test_size` argument specifies the portion of data to sample for testing set.
- The `stratify` argument ensures that portion of data (**malignant** and **benign**) is balanced between training and testing set.

```
In [584]: X = cancer.data  
          y = cancer.target
```

```
In [588]: from sklearn.model_selection import train_test_split  
  
In [589]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=21, stratify=y)  
  
In [590]: len(X_test)  
Out[590]: 171  
  
In [591]: len(X_train)  
Out[591]: 398
```

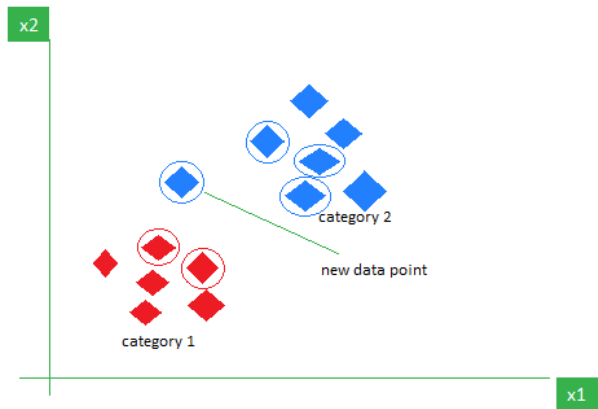
Predicting with scikit-learn



- Most algorithms in scikit-learn use the same implementation procedure.
- You are aware that machine learning process involves two phases: training and testing.
- In scikit-learn, the training phase is executed using the `fit()` method on the chosen algorithm.
- Once you have a trained model, you can use the `predict()` method to predict your test data.

Classification (k-Nearest Neighbors)

- The first classification algorithm that you will explore is the k-Nearest Neighbors (k-NN).
- It is a simple algorithm where it tries to label the target by looking at the label of 'k' closest data points.
- k is an integer > 0 . Hence if $k = 3$, the algorithm looks at the label of 3 nearest data points to the target and take a majority vote. If 2 of the neighbors are 'benign', then the algorithm labels the target as 'benign'
- Analogy: **Are you a handsome man?** 3-NN look at 3 people closest to you. If 2 of them are handsome, then you are handsome too!



Classification (k-Nearest Neighbors)



- Proper consideration is needed when using the k-NN classifier.
- The number of k determines the performance of the model.
- Generally, **larger k creates a less complex model**. This is because it produces smoother decision boundary as it considers more neighbors. (Imagine you taking advice from more people. Your decision should be 'smoother' and more open).
- Choosing a **smaller k can lead to overfitting**. It creates a more complex model.

Classification (k-Nearest Neighbors)

- k-NN can be imported from sklearn.neighbors module.
- You have to instantiate the k-NN object before using it.

```
In [592]: from sklearn.neighbors import KNeighborsClassifier

          knn = KNeighborsClassifier(n_neighbors=3)  # Define the number of neighbors

          knn.fit(X_train, y_train)

Out[592]: KNeighborsClassifier(algorithm='auto', leaf_size=30, metric='minkowski',
                               metric_params=None, n_jobs=1, n_neighbors=3, p=2,
                               weights='uniform')

In [593]: prediction = knn.predict(X_test)
```

Classification (k-Nearest Neighbors)

- You can view the predicted labels by accessing the prediction array.
- To measure your model's performance, you can use the `score()` method.
- `score()` returns the accuracy of your model.
- As you can see, your model recorded 92% accuracy which is not bad after all.

```
In [595]: knn.score(X_test, y_test)
```

```
Out[595]: 0.9239766081871345
```

Classification (Decision Tree)



- The second classification algorithm is the **decision tree**.
- **Decision Trees (DTs)** are a non-parametric supervised learning method used for classification and regression.
- The goal is to create a model that predicts the value of a target variable by learning simple decision rules inferred from the data features.
- A tree can be seen as a piecewise constant approximation.

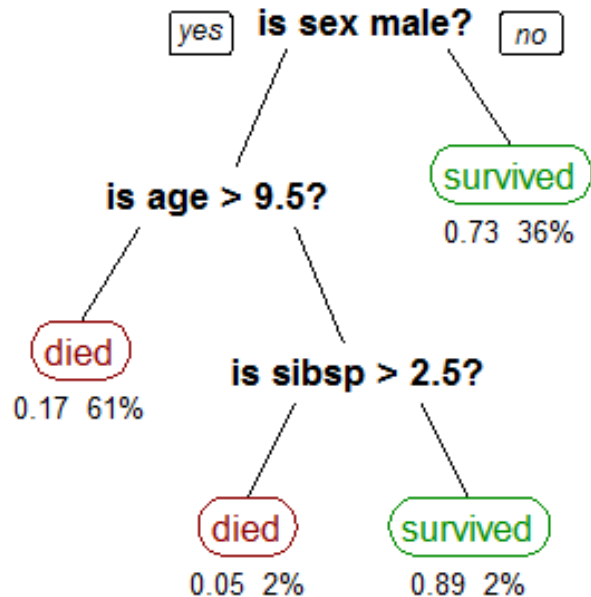
Classification (Decision Tree)



- Key ideas in predicting with trees.
 - Iteratively split variables into groups.
 - Evaluate "homogeneity" within each group.
 - Split again if necessary.
- Pros:
 - Easy to interpret
 - Better performance in nonlinear settings
- Cons:
 - Without pruning/cross-validation can lead to overfitting.
 - Harder to estimate uncertainty.
 - Results may be variable.

Classification (Decision Tree)

- Basic algorithm in trees:
 - Start with all variables in one group.
 - Find the variable/split that best separates the outcomes.
 - Divide the data into two groups (“leaves”) on that split (“node”).
 - Within each split, find the best variable/split that separates the outcomes.
 - Continue until the groups are too small or sufficiently “pure”.



Classification (Decision Tree)

- Accuracy improvement over linear regression

```
from sklearn.datasets import load_iris
from sklearn.model_selection import cross_val_score
from sklearn.tree import DecisionTreeClassifier
```

```
model = DecisionTreeClassifier(random_state=0)
model.fit(X_train, y_train)
model.score(X_test, y_test)
```

```
0.9473684210526315
```

```
model.get_params()
```

```
{'ccp_alpha': 0.0,
 'class_weight': None,
 'criterion': 'gini',
 'max_depth': None,
 'max_features': None,
 'max_leaf_nodes': None,
 'min_impurity_decrease': 0.0,
 'min_samples_leaf': 1,
 'min_samples_split': 2,
 'min_weight_fraction_leaf': 0.0,
 'monotonic_cst': None,
 'random_state': 0,
 'splitter': 'best'}
```

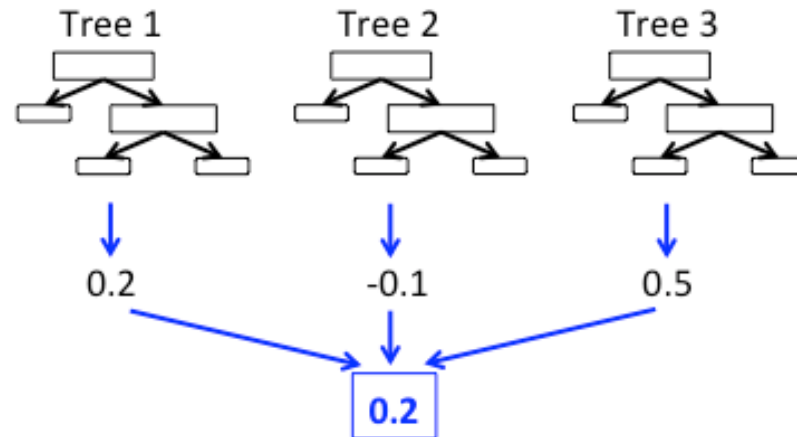
Classification (Random Forest)



- The third classification algorithm is the **random forest**.
- Random forest is an ensemble method. Ensemble methods combine multiple models to create a better final model.
- Random forest is available through the `sklearn.ensemble` module.
- Random forest works by generating a collection of trees. Each of the individual tree is a randomized algorithm. The final prediction is the mode (most frequent) of the classes output by the individual trees.

Classification (Random Forest)

- The basic idea is very similar to bagging in the sense that it bootstrap samples, so it take a resample of the observed data, and the training data set.
- Then it rebuild classification or regression trees on each of those bootstrap samples.
- At each split, bootstrap the variables(resample and recalculate predictions).
- Grow multiple trees and vote.
- Pros:
 - Accuracy
- Cons:
 - Speed
 - Interpretability
 - Overfitting



Classification (Random Forest)

- From the example, random forest **recorded 95% accuracy** which is an improvement if compared to the k-NN algorithm previously.

```
In [597]: from sklearn.ensemble import RandomForestClassifier

/Users/mfaizmzaki/anaconda3/lib/python3.6/importlib/_bootstrap.py:219: RuntimeWarning:
compatibility. Expected 96, got 88
  return f(*args, **kwargs)

In [598]: rf = RandomForestClassifier()

In [599]: rf.fit(X_train, y_train)

Out[599]: RandomForestClassifier(bootstrap=True, class_weight=None, criterion='gini',
    max_depth=None, max_features='auto', max_leaf_nodes=None,
    min_impurity_decrease=0.0, min_impurity_split=None,
    min_samples_leaf=1, min_samples_split=2,
    min_weight_fraction_leaf=0.0, n_estimators=10, n_jobs=1,
    oob_score=False, random_state=None, verbose=0,
    warm_start=False)
```

```
In [601]: prediction_rf = rf.predict(X_test)

In [602]: rf.score(X_test, y_test)

Out[602]: 0.9590643274853801
```

Classification (Support Vector Machines - SVM)



- Support vector machines (SVMs) are a set of supervised learning methods used for classification, regression and outliers detection.
- The advantages of support vector machines are:
 - Effective in high dimensional spaces.
 - Still effective in cases where number of dimensions is greater than the number of samples.
 - Uses a subset of training points in the decision function (called support vectors), so it is also memory efficient.
 - Versatile: different Kernel functions can be specified for the decision function. Common kernels are provided, but it is also possible to specify custom kernels.

Classification (Support Vector Machines - SVM)

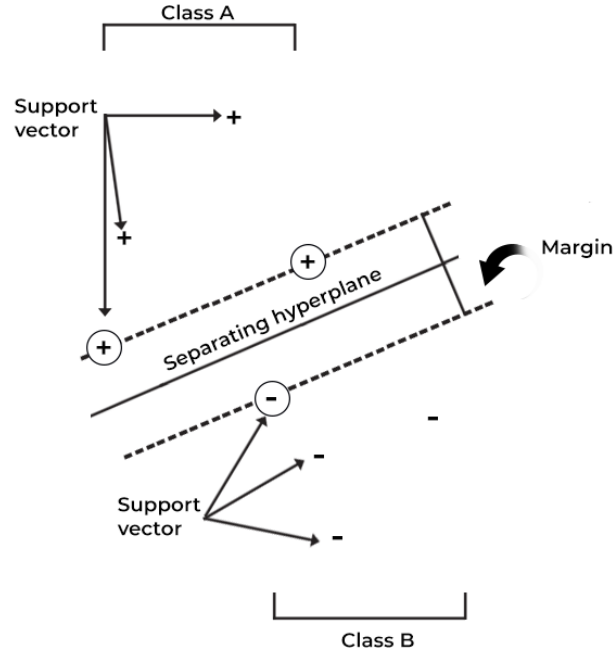


- Technically, the primary objective of the SVM algorithm is to identify a hyperplane that distinguishably segregates the data points of different classes.
- The hyperplane is localized in such a manner that the largest margin separates the classes under consideration.

Classification (Support Vector Machines - SVM)



SVMS OPTIMIZE MARGIN BETWEEN SUPPORT VECTORS OR CLASSES



Classification (Support Vector Machines - SVM)

- How to do it using Sklearn? Easy.

```
from sklearn import svm  
  
model = svm.SVC()  
model.fit(X_train,y_train)  
model.score(X_test,y_test)
```

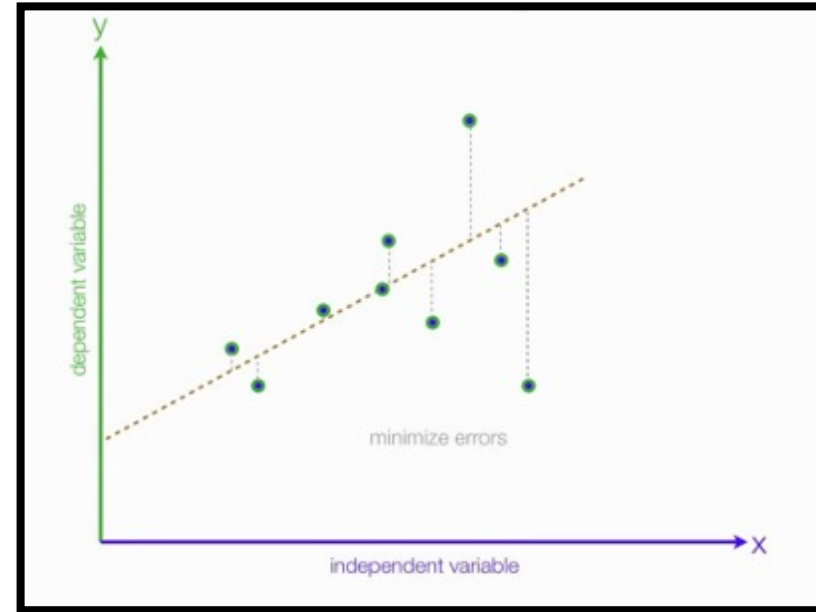
0.935672514619883

```
model.get_params()
```

```
{'C': 1.0,  
 'break_ties': False,  
 'cache_size': 200,  
 'class_weight': None,  
 'coef0': 0.0,  
 'decision_function_shape': 'ovr',  
 'degree': 3,  
 'gamma': 'scale',  
 'kernel': 'rbf',  
 'max_iter': -1,  
 'probability': False,  
 'random_state': None,  
 'shrinking': True,  
 'tol': 0.001,  
 'verbose': False}
```

Regression (Linear Regression)

- Simple Linear Regression (SLR) is a statistical method that examines the linear relationship between two continuous variables, X and Y.
- X is regarded as the independent variable while Y is regarded as the dependent variable. SLR discovers the best fitting line using **Ordinary Least Squares (OLS)** criterion.
- OLS criterion minimizes the sum of squared prediction error.
- Prediction error is defined as the difference between actual value (Y) and predicted value ($\hat{Y}$) of dependent variable. **OLS** minimizes the squared error.



Regression (Linear Regression)



- For regression problem, you will be using the **Boston House dataset** from sklearn datasets.
- Linear regression is a basic regression algorithm.
- It predicts a continuous value by fitting a best-fit line (minimizing squared error) across all data points.

Regression (Linear Regression)

Use the housing price dataset from Sklearn

```
from sklearn import datasets
```

```
house = datasets.fetch_california_housing()
```

```
X = house.data
```

```
y = house.target
```

```
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.3)
```

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
```

```
model.fit(X_train,y_train)
```

```
prediction = model.predict(X_test)
```

```
model.score(X_test,y_test)
```

```
0.592572799783585
```

Regression (Linear Regression)



- Unlike classification, the `score()` method in regression calculates **the R-squared** value instead of accuracy.
- R-square measures **how close the data are** to the fitted regression line.
- The score ranges from 0 to 1. The higher the R-squared, the better the model fits your data.
- From the score, the regression model scored 59%. You can improve on this score by optimizing your model e.g. feature selection, cross validation.

Regression (Linear Regression)



- It is useful if you plot the regression line against your test data to observe how it fits.
- However, based on the previous example, it is difficult to plot as the columns are more than 2 dimension.
- To make it possible, you should try fitting the model with only one variable.
- In our case, let's try and fit it against the median income per household.
- You can extract the median income per household column (column 1) using square brackets i.e. `house['data'][:, 1]`

Regression (Linear Regression)

- The figure below shows our regression line against one variable: Median income
- The `prediction_lin` variable is the prediction results on the median incomes.
- The `y_test` variable is the true values on the number of rooms.

```
from sklearn.linear_model import LinearRegression

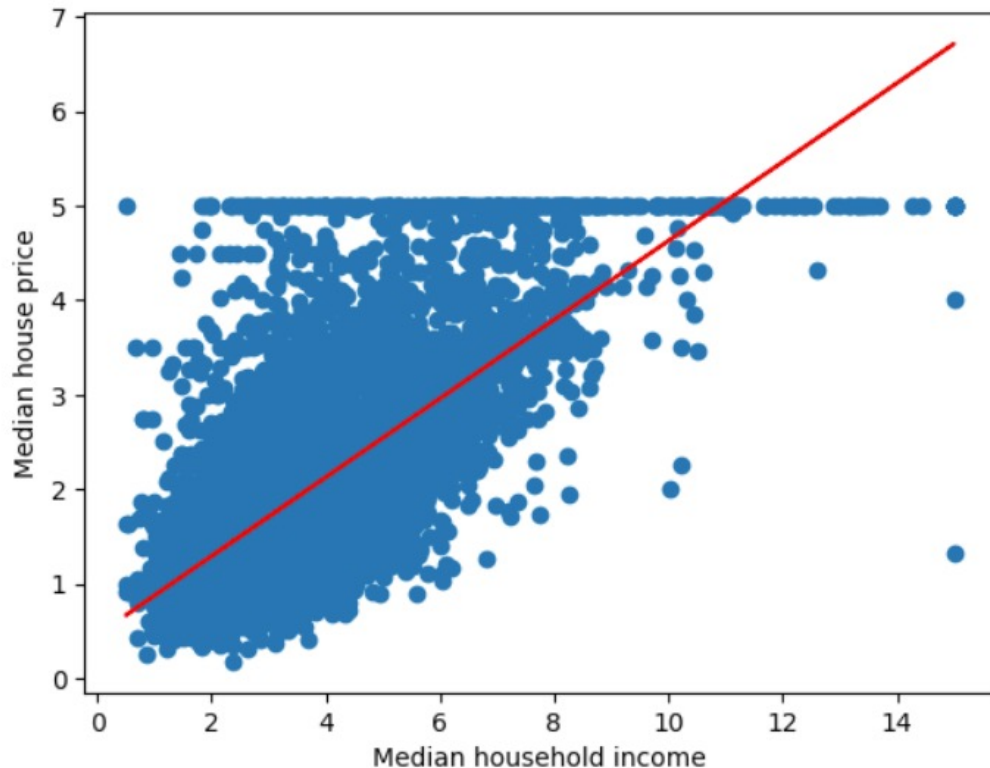
model = LinearRegression()
model.fit(X_train[:,0].reshape(-1, 1), y_train)
prediction = model.predict(X_test[:,0].reshape(-1, 1))
model.score(X_test[:,0].reshape(-1, 1), y_test)
```

0.47762812729084037

Regression (Linear Regression)

```
import matplotlib.pyplot as plt

plt.scatter(X_test[:,0],y_test)
plt.plot(X_test[:,0],prediction, color='red')
plt.xlabel('Median household income')
plt.ylabel('Median house price')
```



Regression (Ridge Regression)



- Ridge regression is a regularized regression.
- Remember a regular regression tries to draw a best fit line by minimizing the squared errors.
- The algorithm chooses a coefficient for each feature variable i.e.

$$y = a_1x_1 + a_2x_2 + \dots + a_nx_n + b \text{ where } a_n \text{ are the coefficients.}$$

- Large coefficients can lead to overfitting.
- Ridge regression penalizes large coefficients. In other words, it regularizes the coefficients.

Regression (Ridge Regression)

- Ridge regression recorded 59.26% r-squared.
- Similar to the regular regression.
- This model can improve by tweaking and optimization.

```
from sklearn.linear_model import Ridge
```

```
model = Ridge(alpha=0.1)  
model.fit(X_train,y_train)  
prediction = model.predict(X_test)  
model.score(X_test,y_test)
```

```
0.5925792457551244
```

```
model.get_params()
```

```
{'alpha': 0.1,  
 'copy_X': True,  
 'fit_intercept': True,  
 'max_iter': None,  
 'positive': False,  
 'random_state': None,  
 'solver': 'auto',  
 'tol': 0.0001}
```

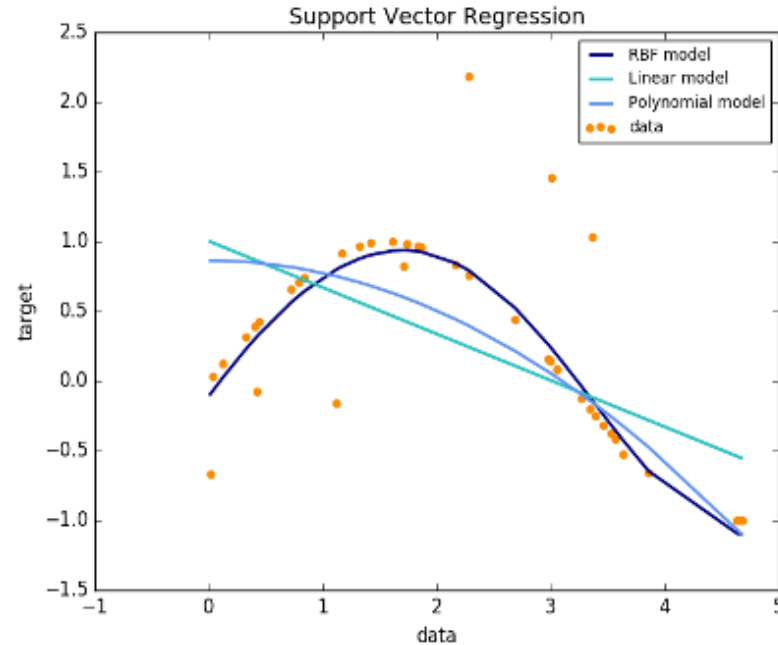
Regression (Ridge Regression)



- This algorithm introduces the parameter α . α puts a constraint on the coefficients so they don't get too large.
- Setting $\alpha = 0$ will give you a regular linear regression as before.
- Setting higher values of α reduces overfitting until some extent that it induces underfitting.
- Determining the value α is an optimization problem.

Regression (Support Vector Regression)

- Support Vector Regression (SVR) works on similar principles as Support Vector Machine (SVM) classification.
- One can say that SVR is the adapted form of SVM when the dependent variable is numerical rather than categorical.
- Major benefit of using SVR:
 - It is a non-parametric technique.
 - SVR technique depends on kernel functions.



Regression (Support Vector Regression)

- How to implement in sklearn?
- Bad performance. So what can you do:
 - Change kernel function
 - Tune the hyper parameters
 - Features engineering

```
from sklearn import svm  
  
model = svm.SVR()  
model.fit(X_train,y_train)  
model.score(X_test,y_test)
```

-0.02905609872619408

```
model.get_params()
```

```
{'C': 1.0,  
 'cache_size': 200,  
 'coef0': 0.0,  
 'degree': 3,  
 'epsilon': 0.1,  
 'gamma': 'scale',  
 'kernel': 'rbf',  
 'max_iter': -1,  
 'shrinking': True,  
 'tol': 0.001,  
 'verbose': False}
```

Cross Validation



- A model's performance is highly dependent on the way the data is split.
- The portion between training and testing sets affect the final output.
- Therefore, it is not representative of the model's ability to generalize.
- As a solution, data scientist usually implement cross validation.
- Cross validation increases the amount of available data and allows averaging of the errors.

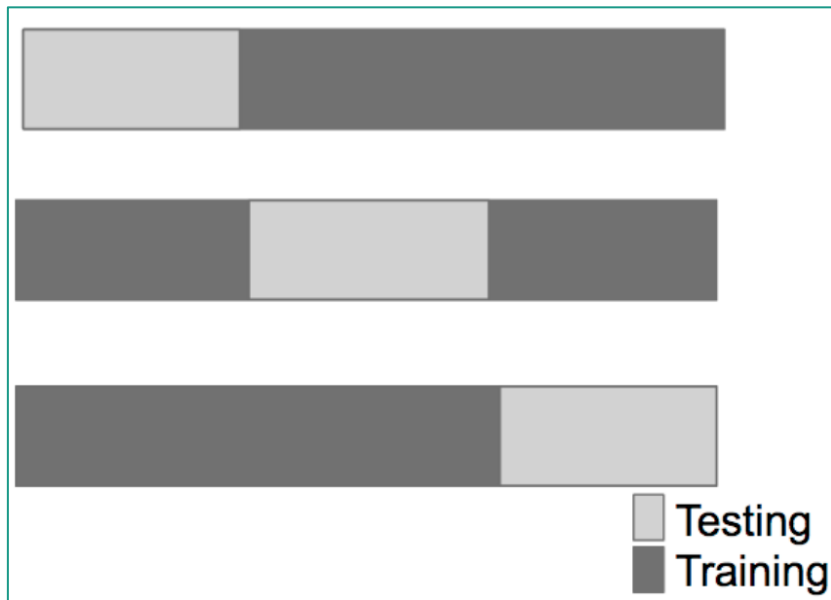
Cross Validation



- There are various approaches to cross validation. Among them are:
 - K-Fold
 - Random Sampling
 - Leave-One-Out
- All the approaches hold the same concept where they iteratively split the dataset into two portion: training and testing.

K-Fold Cross-Validation

- It splits the dataset into **equal** 'k'-folds.
- Then, the model combines k-1 folds as the training set and uses the remaining 1 fold as the testing set.
- This process repeats for k times by rotating the test set between the folds.
- In the end, you will have k different metrics and can take the average.



K-Fold Cross-Validation



Split 1	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Metric 1
Split 2	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Metric 2
Split 3	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Metric 3
Split 4	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Metric 4
Split 5	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Metric 5

Training data

Test data

K-Fold Cross-Validation

- A test run using cross validation on the classification problem you have done earlier showed slight improvement in the accuracy score.

```
In [595]: knn.score(X_test, y_test)
```

```
Out[595]: 0.9239766081871345
```

Without cross-validation

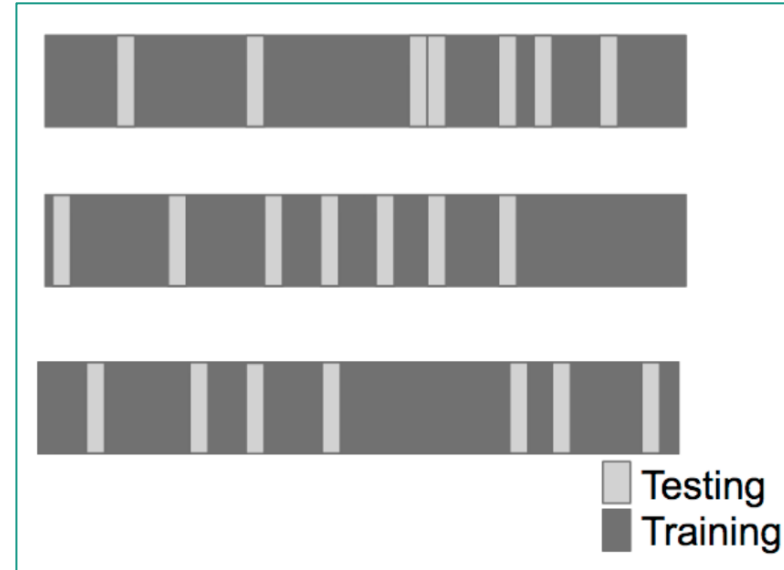
```
In [692]: from sklearn.model_selection import cross_val_score  
cv_results = cross_val_score(knn, X, y, cv=10)  
np.mean(cv_results)
```

```
Out[692]: 0.9263330740644717
```

With 10-fold cross-validation

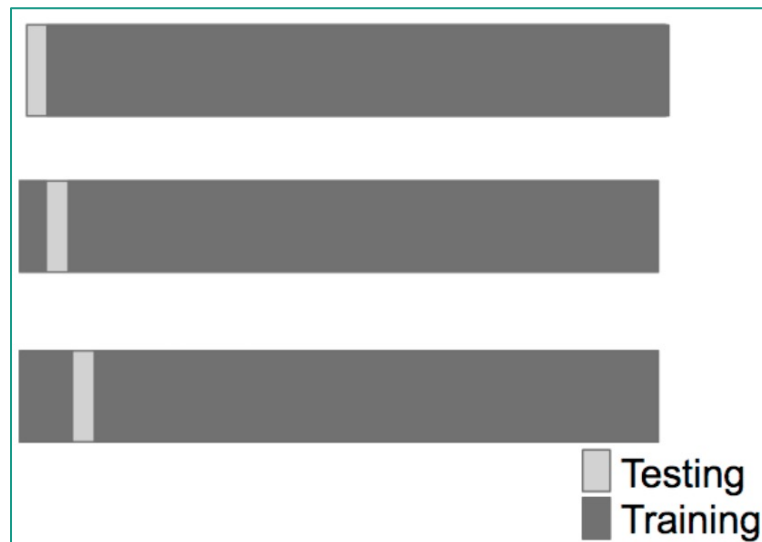
Random Sampling Cross-Validation

- Using random sampling, the algorithm randomly selects a defined portion of the data as the training set and uses the remaining as the test set.
- The process repeats for a finite amount of iterations.
- In random sampling, some of the data may be reappear in different iterations.
- The average of the results is taken as final output.



Leave-One-Out Cross-Validation

- Leave-One-Out is another common approach of cross validation.
- The idea is simple: Take the entire dataset EXCEPT one sample as the training set. Use that remaining one sample as the testing set.
- Iterate through the entire dataset.
- This algorithm is really expensive. If you have big data, Leave-One-Out may not be your best option.



Considerations in Cross-Validation



- For k-fold cross validation:
 - Larger k = less bias, more variance
 - Smaller k = more bias, less variance
- Random sampling must be done without replacement.
- Random sampling with replacement is called bootstrap.
 - Underestimates the error.
 - Can be corrected but complicated.
- If cross-validate to pick predictors estimate, you must estimate errors on independent data.